

Introduction

Computer Architecture (ME-5319)

Phung Thanh Huy

Faculty of Mechanical Engineering
Ho Chi Minh City University of Technology (HCMUT) - VNUHCM

2026-v1



References and Learning Resources

Software

- Logisim Evaluation (Logic Analysis and Design)
- PlatformIO (inside VSCode)
- Intel Quartus
- Venus RISC-V

FPGA

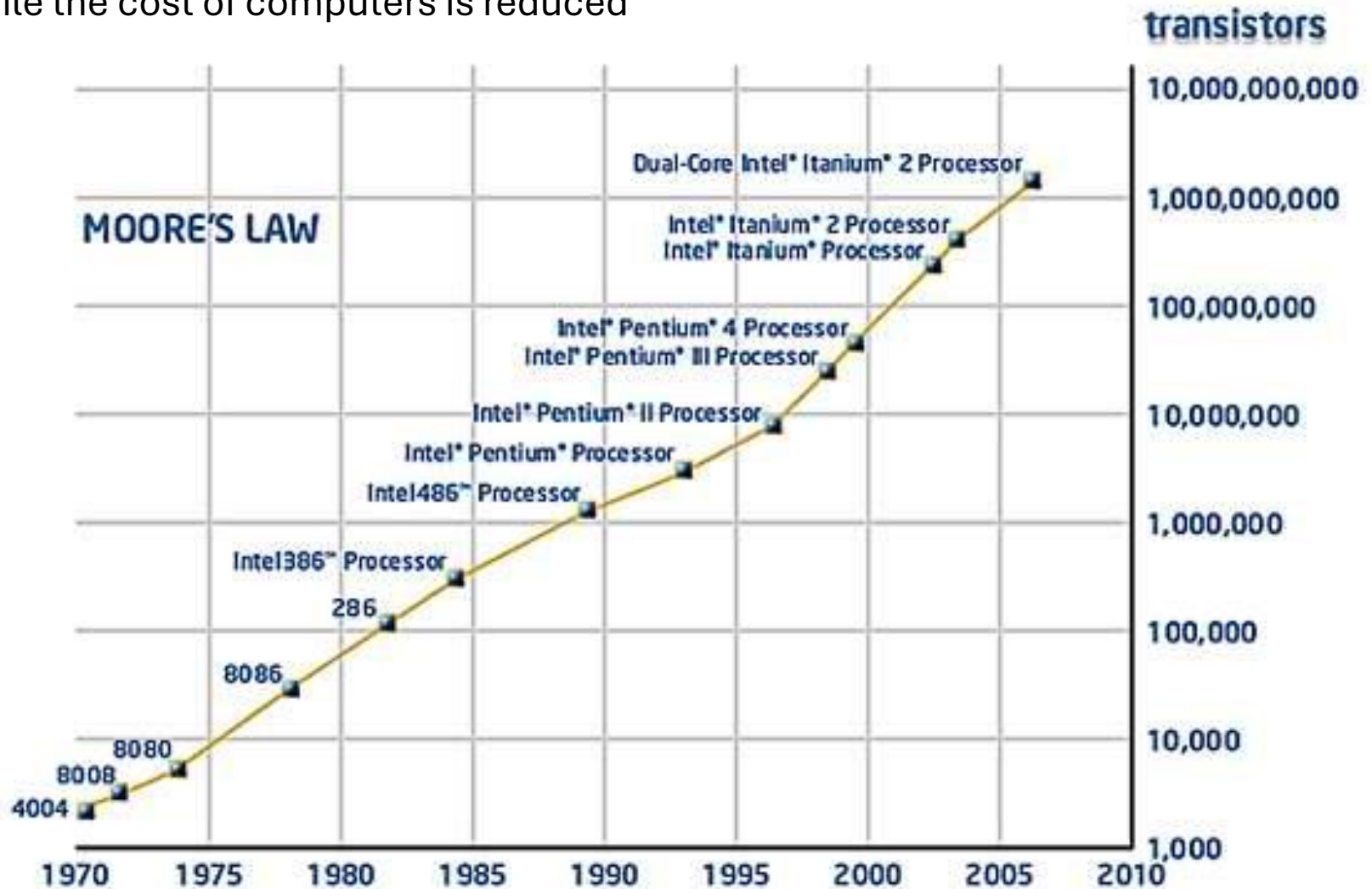
- Xilinx - Altera (if possible)
- Tang nano 4k, 9k, 20k (low cost)
- Zynq board...



Introduction to Computer Architecture

The Moore's Law (Empirical law)

- The number of transistors on a microchip doubles approximately every two years while the cost of computers is reduced



Introduction to Computer Architecture

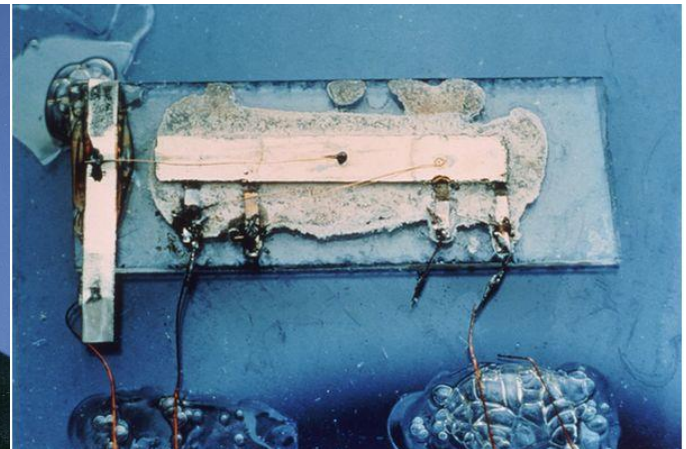
Computer Revolution

The first generation

- Vacuum tubes
- 1946 – 1955

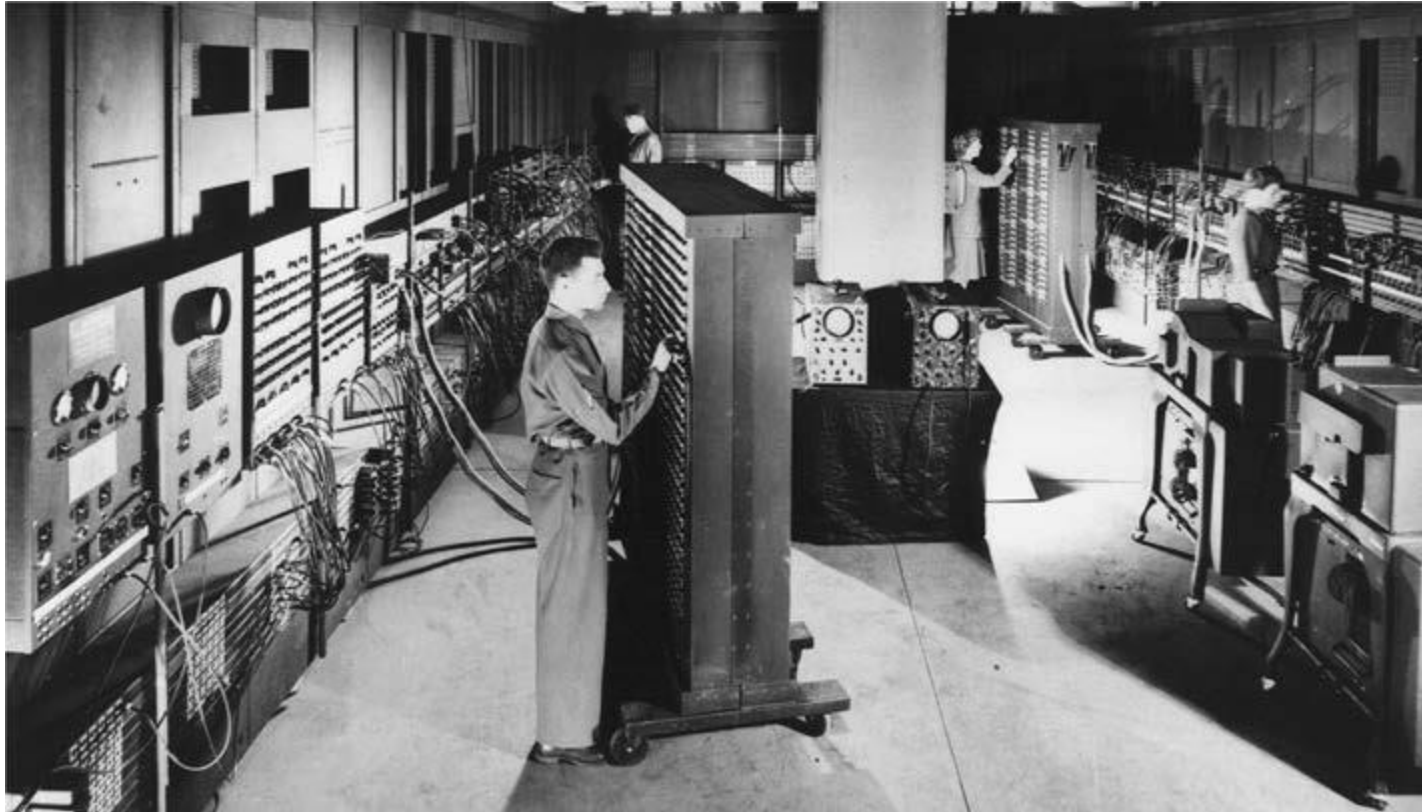
The second generation

- Transistors
- 1955 – 1965



Introduction to Computer Architecture

Computer Revolution



ENIAC - 1946

Introduction to Computer Architecture

Computer Revolution

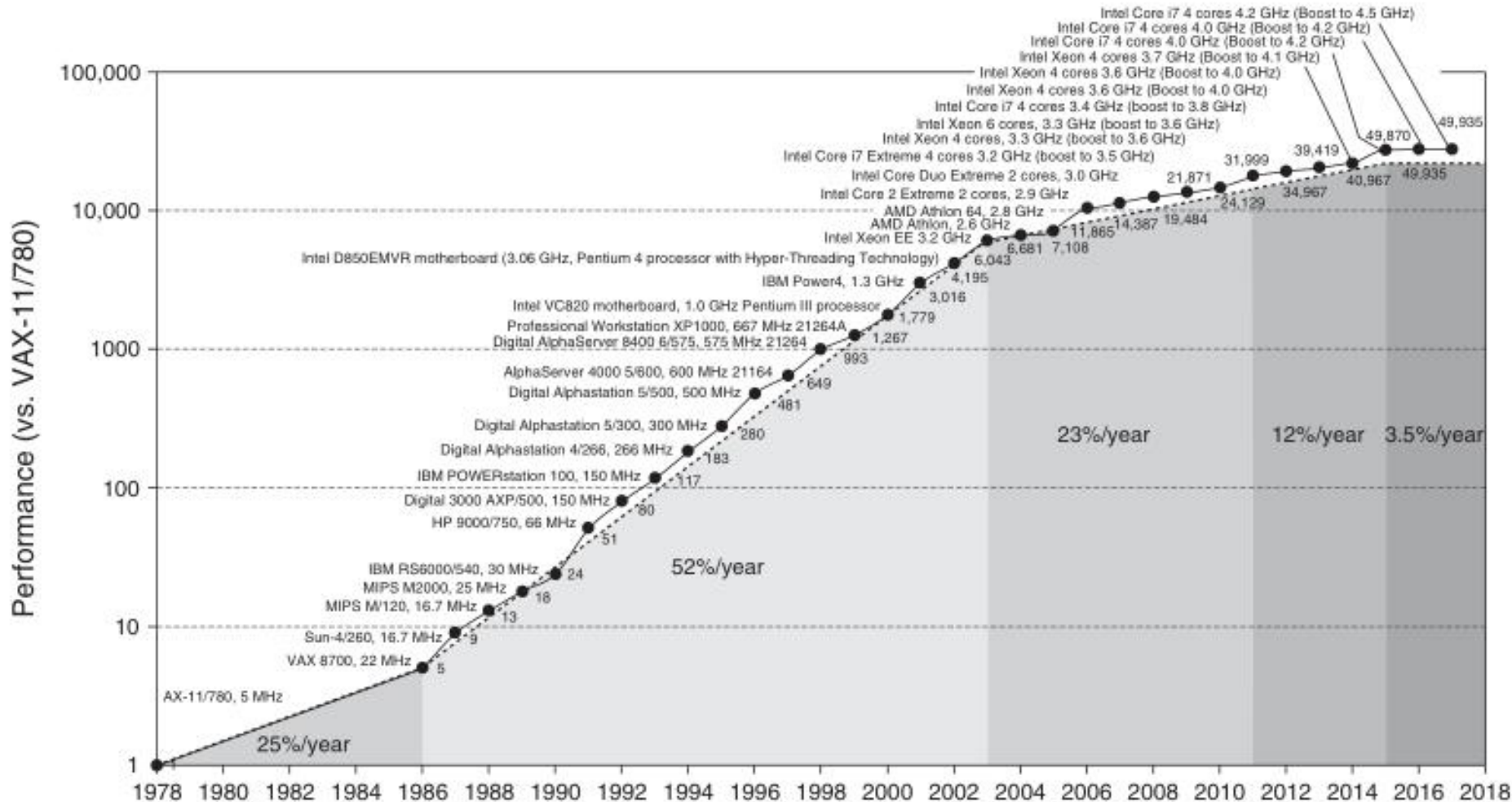


UNIVAC I - 1952

Introduction to Computer Architecture

Computer Revolution

- The Moore's Law is reaching the limit!



Introduction to Computer Architecture

What's next?



Supper computer

Introduction to Computer Architecture

What's next?



<https://www.sci.news/othersciences/computerscience/qubit-copies-14467.html>

Introduction to Computer Architecture

What's next?

Parallel and Tensor/ Neural computing

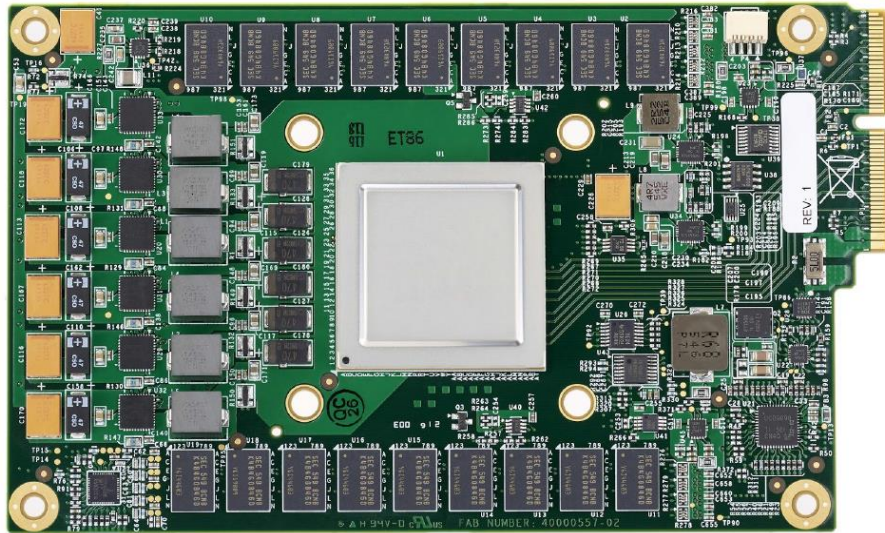


Figure 3. TPU Printed Circuit Board. It can be inserted in the slot for an SATA disk in a server, but the card uses PCIe Gen3 x16.

Credit: Prof. Onur Mutlu

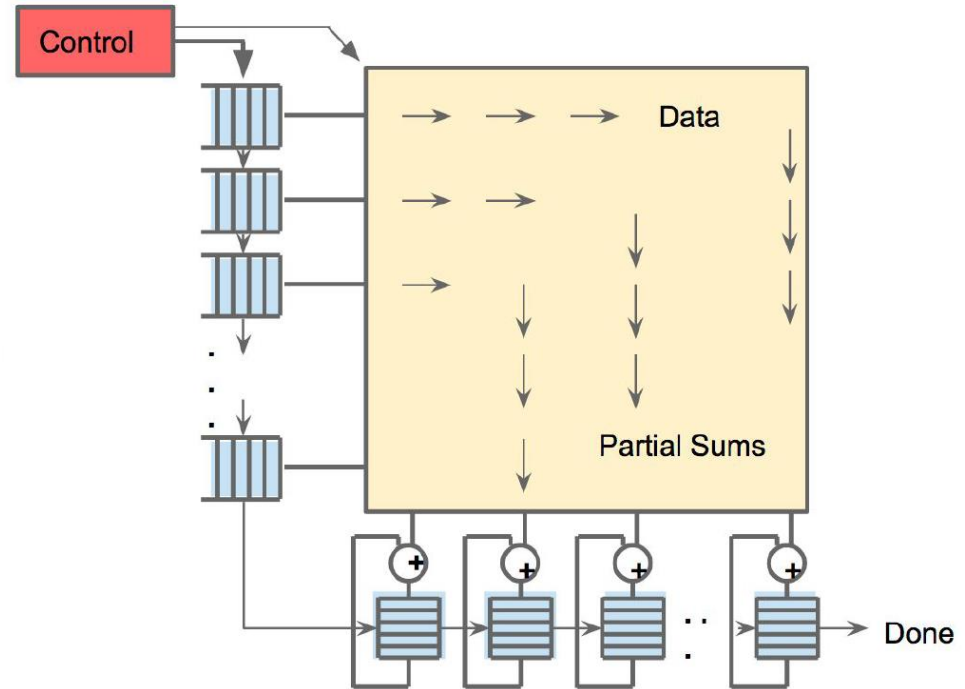


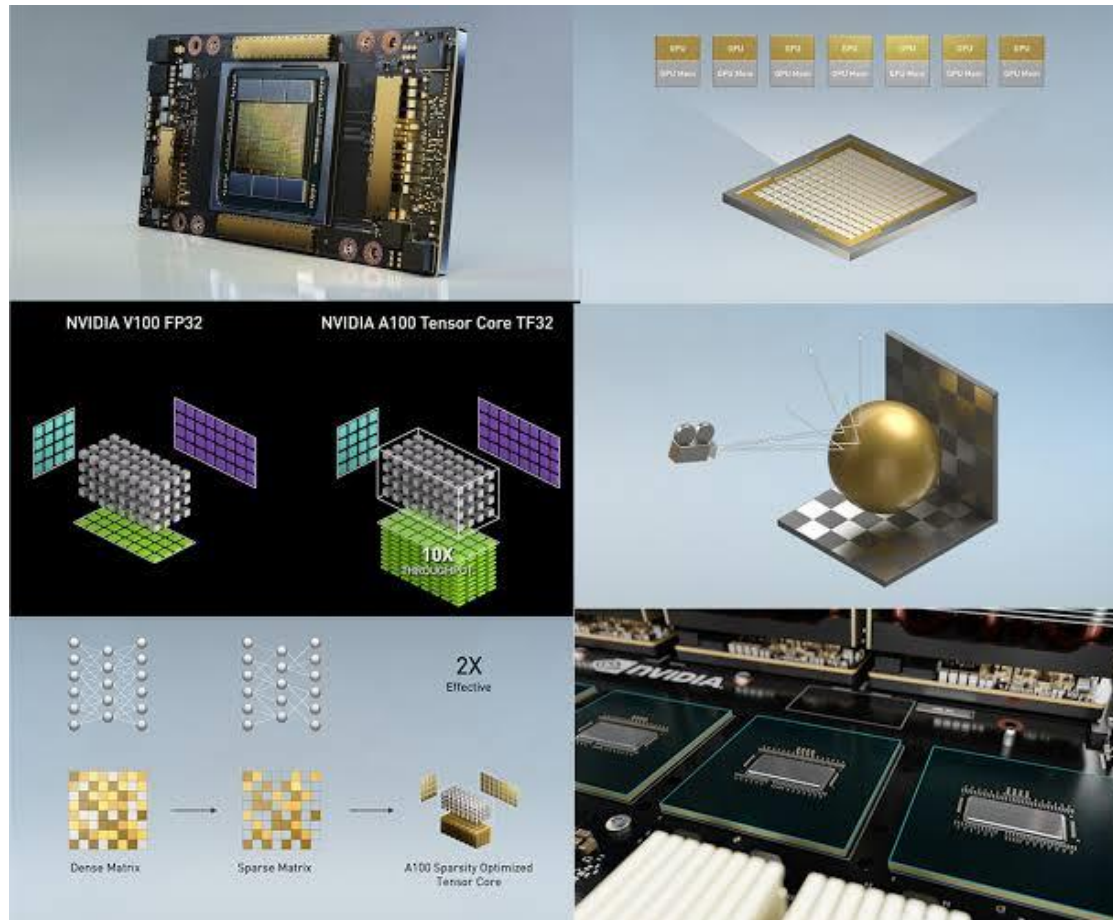
Figure 4. Systolic data flow of the Matrix Multiply Unit. Software has the illusion that each 256B input is read at once, and they instantly update one location of each of 256 accumulator RAMs.

Jouppi et al., “In-Datcenter Performance Analysis of a Tensor Processing Unit”, ISCA 2017.

Introduction to Computer Architecture

What's next?

Parallel and Tensor/ Neural computing



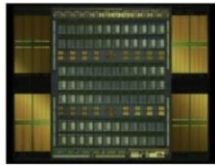
<https://blogs.nvidia.com/blog/what-is-cuda-2/>

Introduction to Computer Architecture

What's next?

Credit: Prof. Onur Mutlu

Parallel and Tensor/ Neural computing



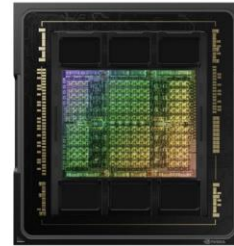
Volta

>21 billion transistors
815mm²
TSMC 12nm FFN



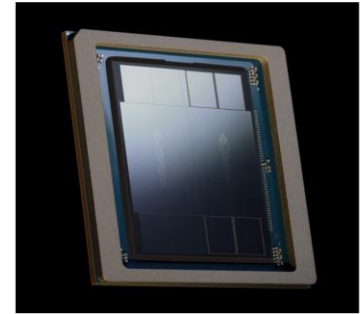
Ampere

>54 billion transistors
826 mm²
TSMC N7



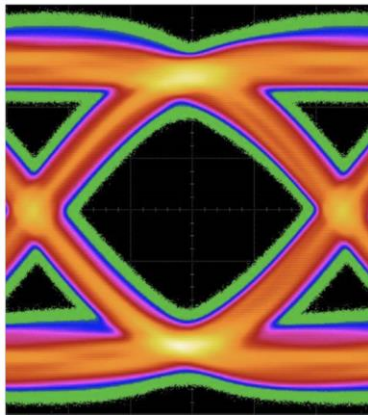
Hopper

>80 billion transistors
814 mm²
TSMC 4N

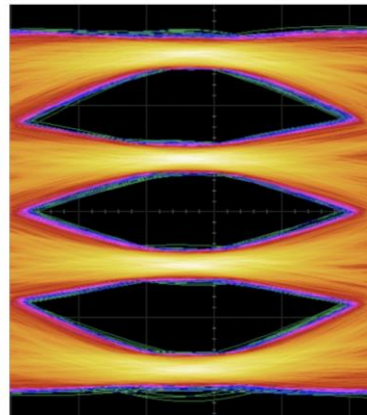


Blackwell

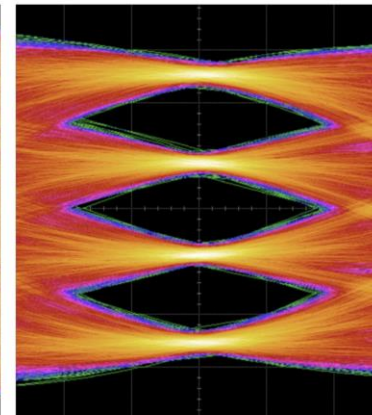
>208 billion transistors
>1600 mm²
TSMC 4NP



Ampere | NVLink3
12 NVLinks | 50GB/s each
x4@50Gbps-NRZ
600GB/s total



Hopper | NVLink4
18 NVLinks | 50GB/s each
x2@100Gbps-PAM4
900GB/s total

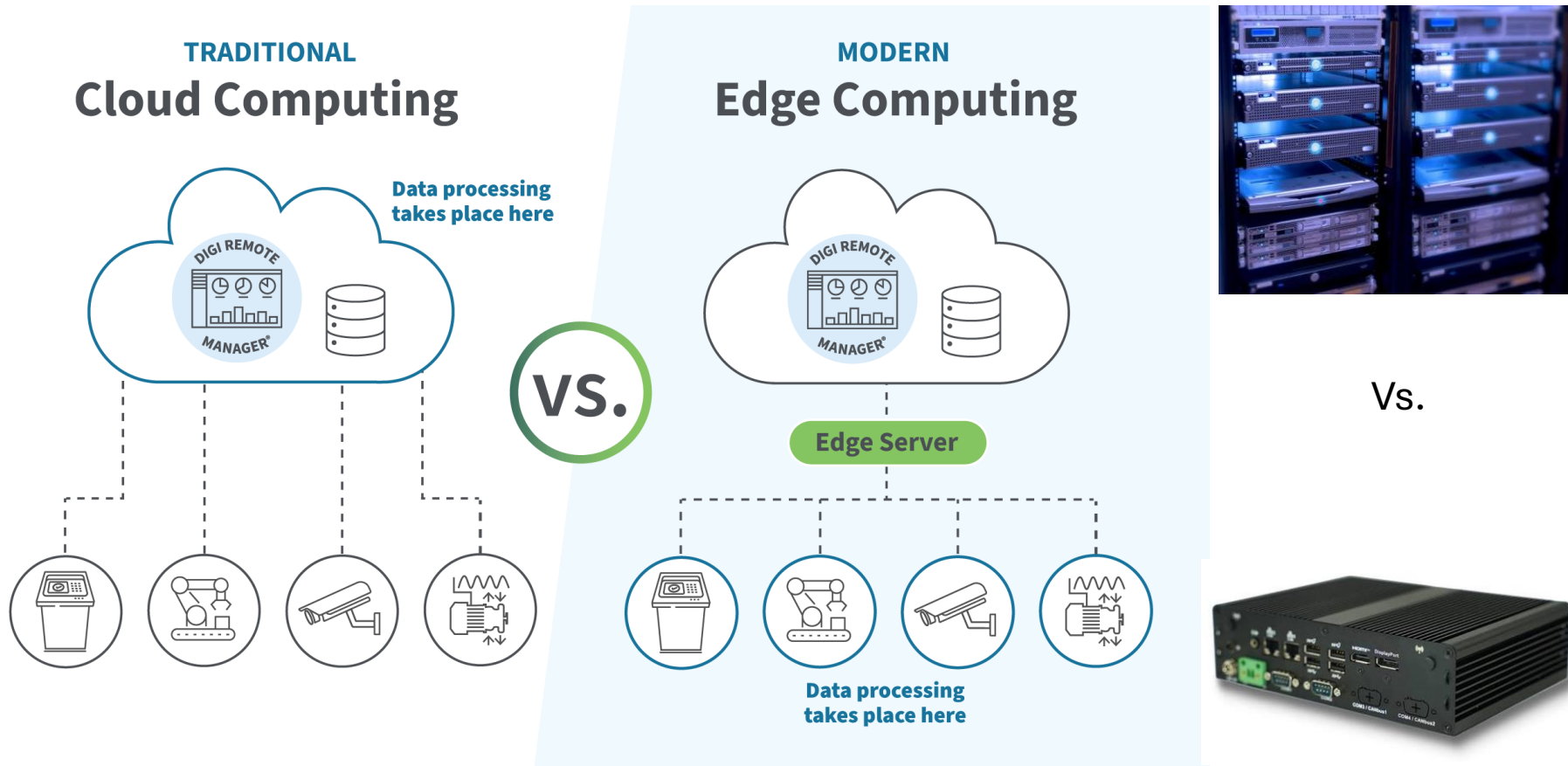


Blackwell | NVLink5
18 NVLinks | 100GB/s each
x2@200Gbps-PAM4
1800GB/s total

Introduction to Computer Architecture

What's next?

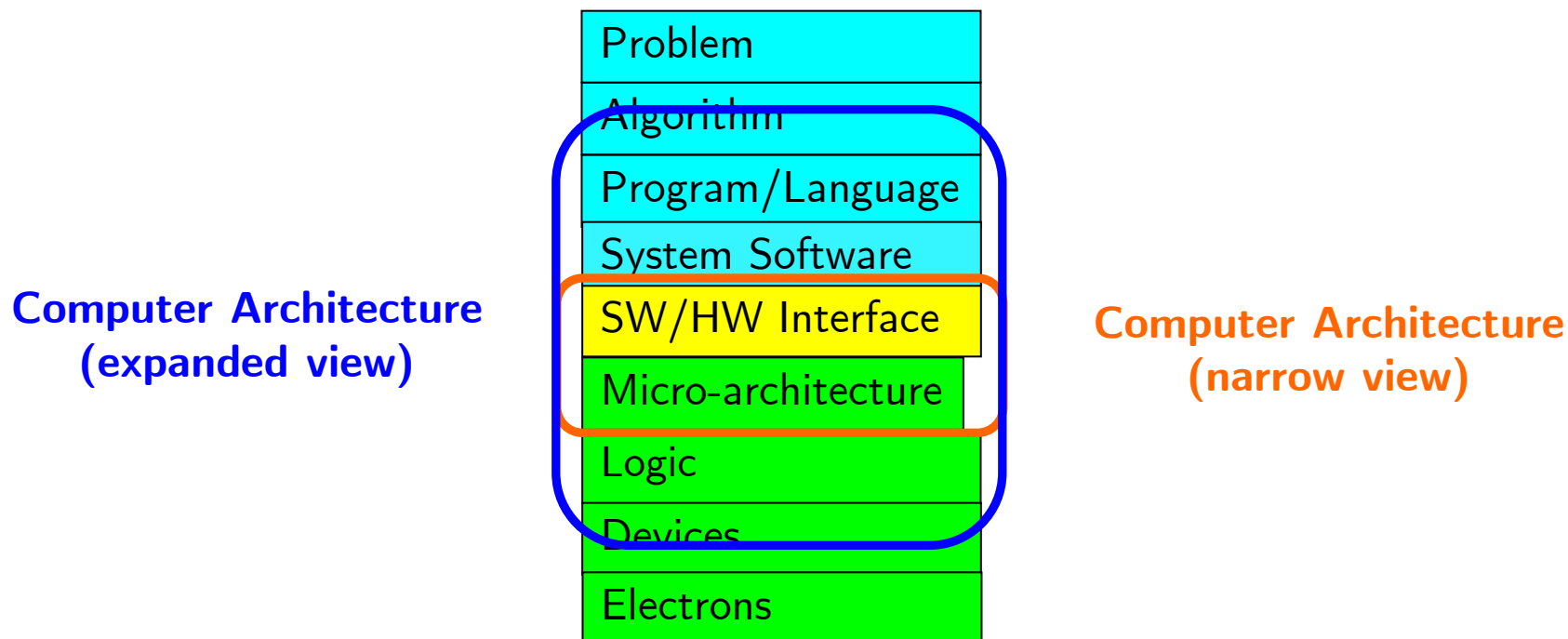
Cloud and Edge



<https://blogs.nvidia.com/blog/what-is-cuda-2/>

Introduction to Computer Architecture

Computer Architecture Overview



Credit: Prof. Onur Mutlu

Introduction to Computer Architecture

Computer Architecture Overview

Algorithm

Step-by-step procedure that is **guaranteed to terminate** where **each step is precisely stated** and can be carried out by a computer

- **Finiteness**
- **Definiteness**
- **Effective computability**

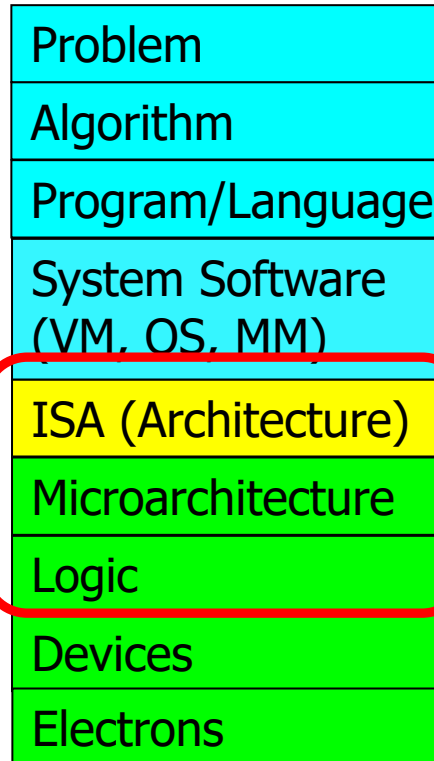
Many algorithms for the same problem

Microarchitecture

An implementation of the ISA

Digital logic circuits

Building blocks of micro-arch (e.g., gates)



ISA

(Instruction Set Architecture)

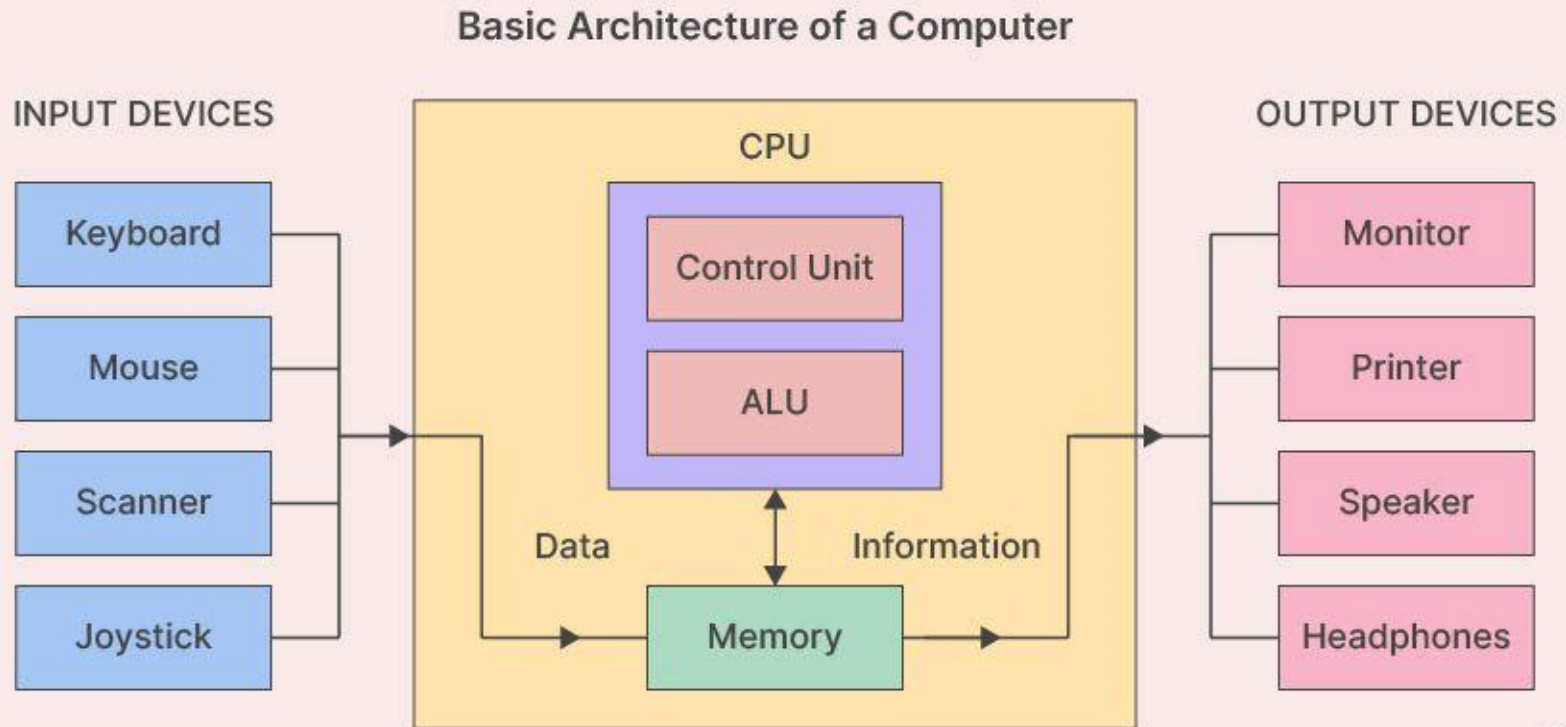
Interface/contract between SW and HW.

What the programmer assumes hardware will satisfy.

Credit: Prof. Onur Mutlu

Introduction to Computer Architecture

Basic Structure of a Computer



Introduction to Computer Architecture

Devices

Credit: Prof. Onur Mutlu

Sensors

SoC
with lots of
compute
& caches

Storage

Main Memory

Main Memory

Storage

Apple M1 Ultra System (2022)

Introduction to Computer Architecture

Devices

Credit: Prof. Onur Mutlu



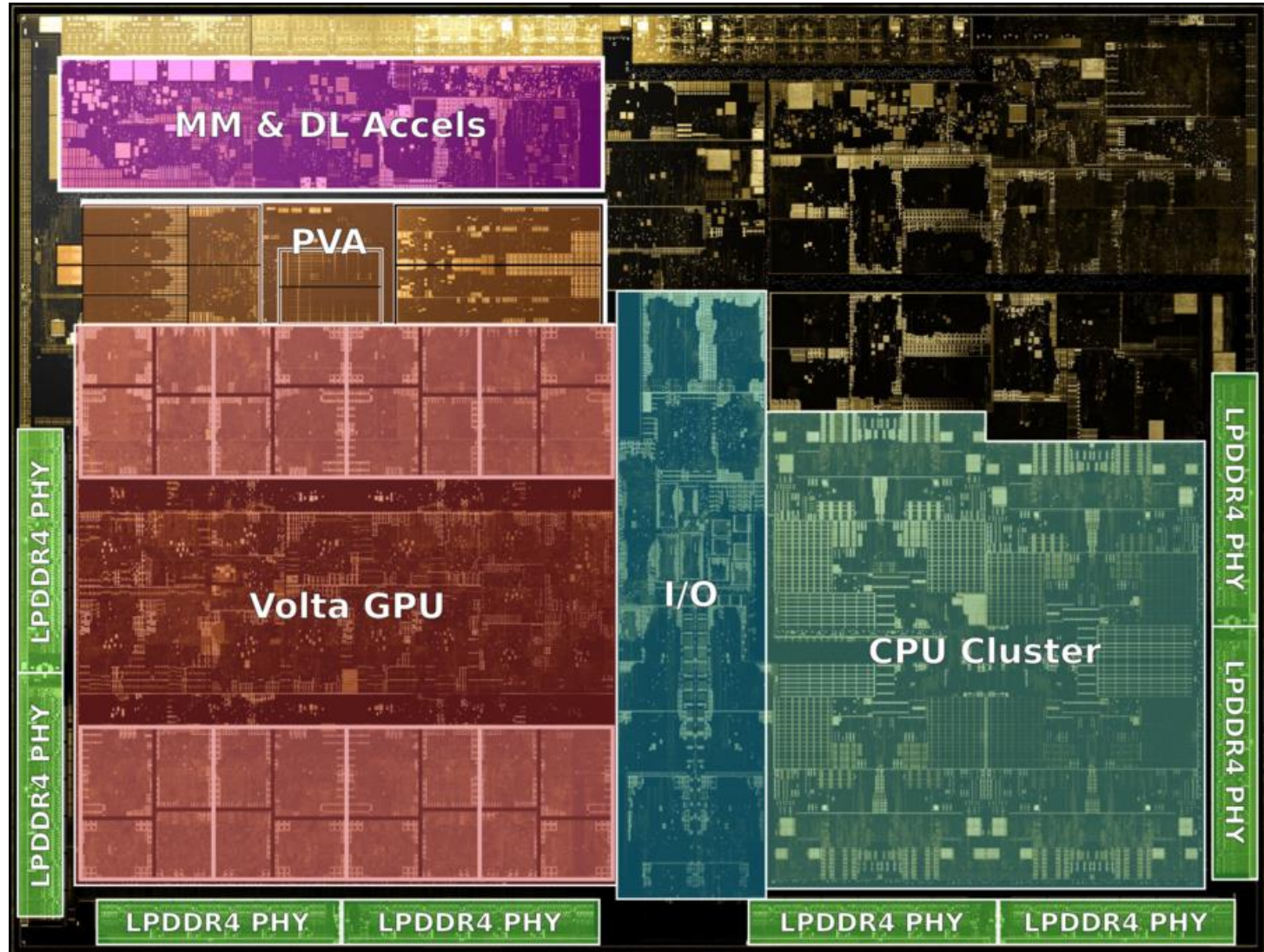
10nm ESF=Intel 7 Alder Lake die shot (~209mm²) from Intel: <https://www.intel.com/content/www/us/en/newsroom/news/12th-gen-core-processors.html>
Die shot interpretation by Locuza, October 2021

Intel Alder Lake, 2021

Introduction to Computer Architecture

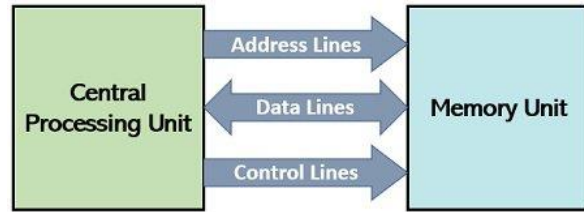
Devices

Credit: Prof. Onur Mutlu

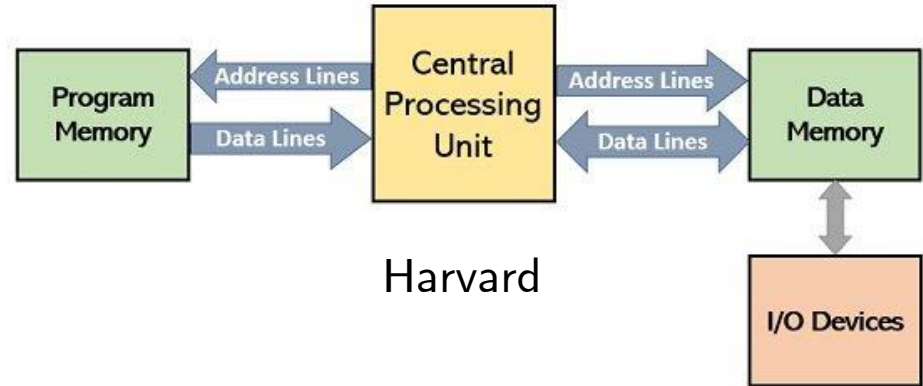


Introduction to Computer Architecture

Architecture



Princeton (Von Neumann)



Harvard

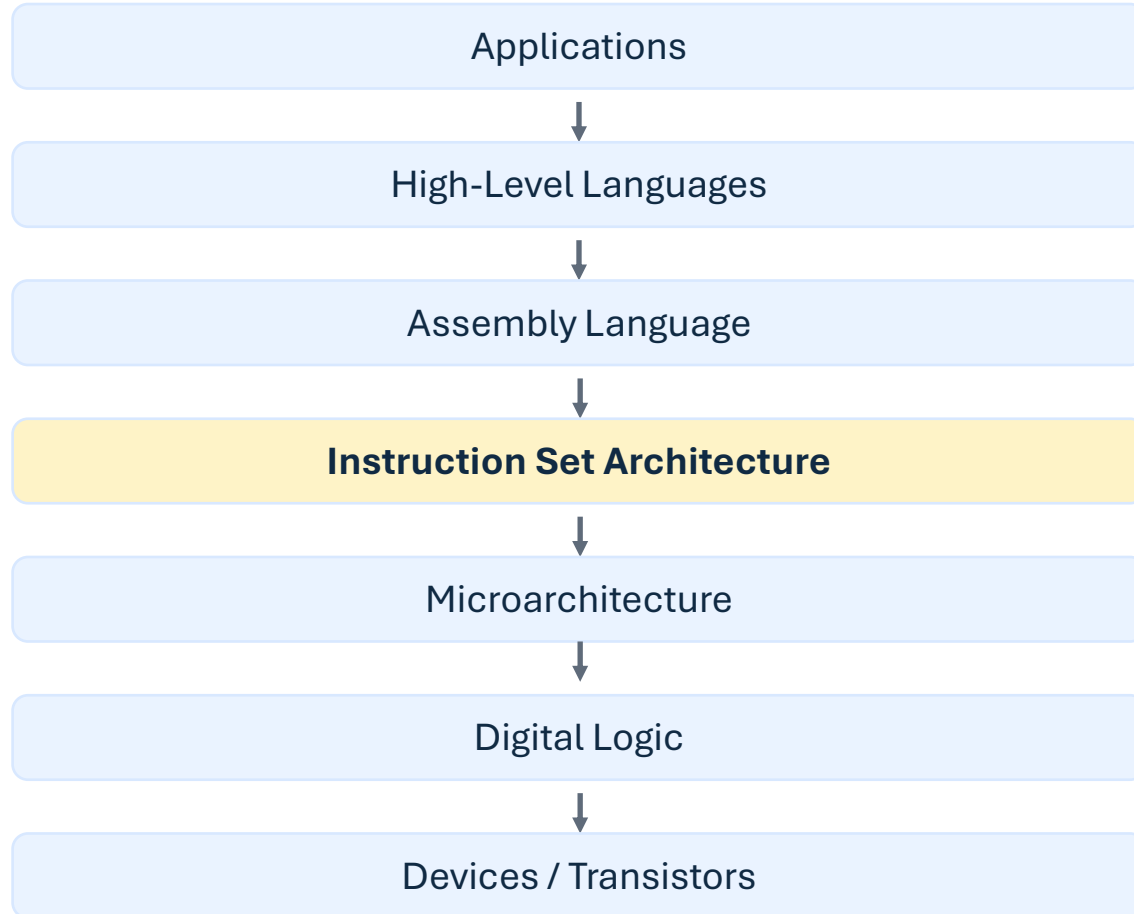
ISA – Instruction Set Architecture

- MIPS
- RISC,
- RISC-V
- ARM
- ...

Introduction to Computer Architecture

Abstraction Layers

Software



Hardware

```
c = a + b;
```

```
add x5, x6, x7
```

```
0000000 00111 00110  
000 00101 0110011
```


Introduction to Computer Architecture

Abstraction Layers

ISA

- What does Processor do?
- What does Software see?
 - Eg: add x5,x6,x7

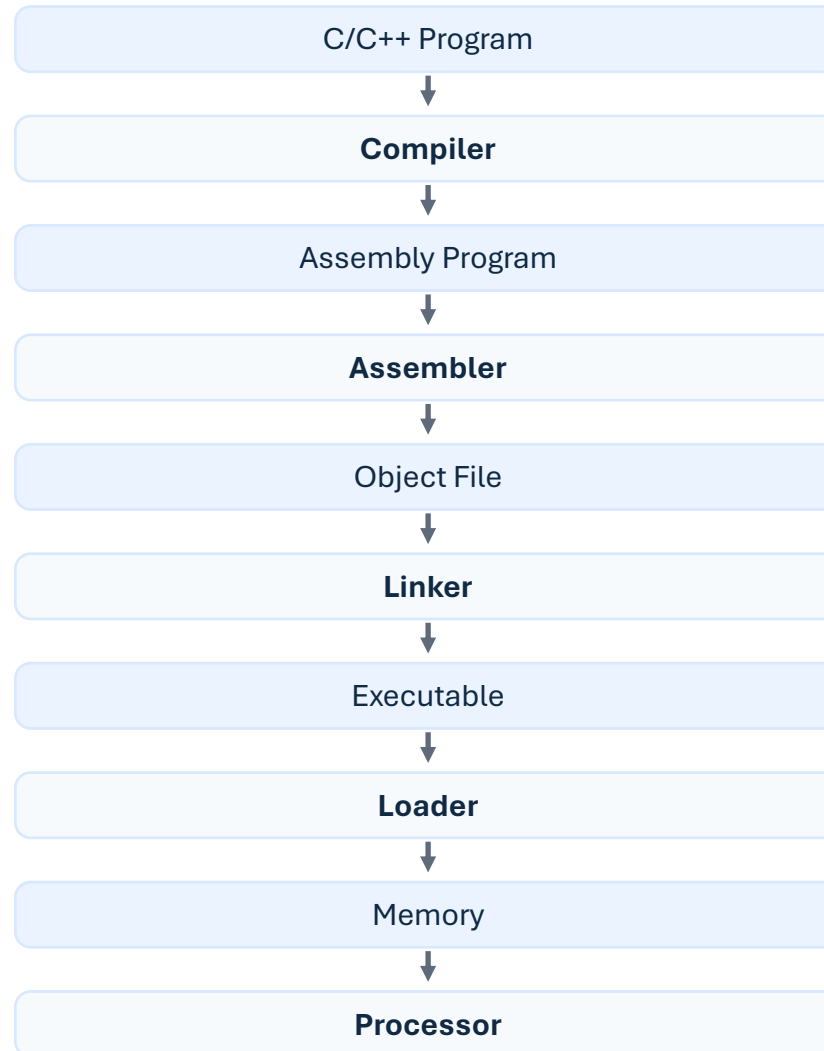
Microarchitecture

- How does processor do?
- What is inside a processor
- Single-cycle, pipeline, superscalar

Many processor can implement the same ISA.

ISA and Computer workflow

A simple C++ program



ISA and Computer workflow

Compiler

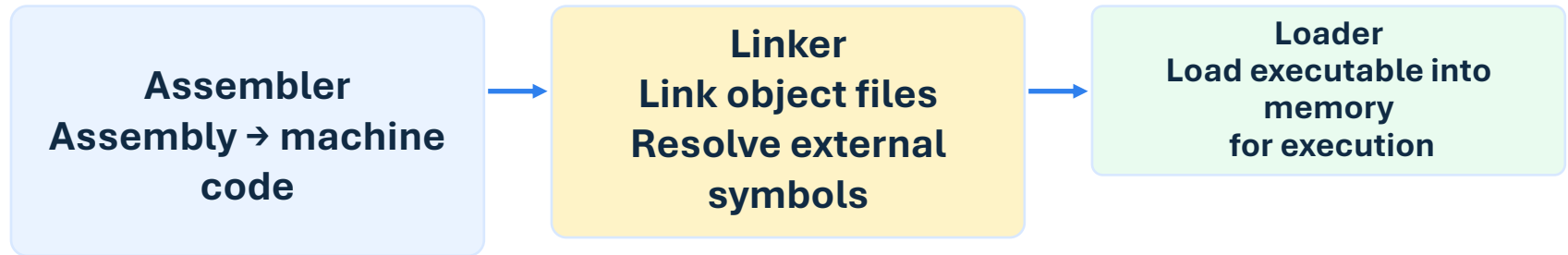
- Translates the code into low-level code.
- Optimize the code
- Generate the instruction according to ISA
- The same source code can generate different machine code for RISC-V, ARM, x86

```
int add(int a, int b) {  
    return a + b;  
}
```

```
// can be come:  
add a0, a0, a1  
ret
```


ISA and Computer workflow

Assembler , Linker, Loader



CPU only starts when the executable is loaded into memory.

Performance of a Computer

Abstraction Layers

Processor A
4.0 GHz

Processor B
3.2 GHz

Is A “faster” than B?

Performance of a Computer

Common Perspectives

Response Time / Latency

Time required to complete one task.

Throughput

Number of tasks completed per unit time.

- Example:
 - Laptop application → latency matters.
 - Web server → throughput often matters.

Performance of a Computer

Common Perspectives

Criteria	Response Time (Latency / Execution Time)	Throughput (Bandwidth)
<i>Definition</i>	The total time required to complete a single task from start to finish.	The total amount of work (number of tasks) the system completes in a given unit of time.
<i>Units of Measurement</i>	Seconds (s), milliseconds (ms), clock cycles.	Tasks/second, Instructions/second (MIPS, FLOPS), Bytes/second (GB/s).
<i>Optimization Goal</i>	Finishing a specific job as fast as possible.	Packing and processing as many jobs as possible simultaneously.
<i>Hardware Techniques</i>	Increasing clock frequency (F_{max}), using high-speed Caches, optimizing the critical path.	Pipelining, multi-core processors, parallel execution units.
<i>Prioritized Systems</i>	Personal computers, real-time embedded systems running closed-loop control algorithms (e.g., Sliding Mode Control).	Cloud servers, data centers, AI hardware accelerators processing batch workloads.
<i>Practical Example</i>	Upgrading to a CPU with a higher clock speed compiles a C/C++ file faster.	Adding more cores to a server allows it to serve thousands of users concurrently, even if the processing time per user is unchanged.

Performance of a Computer

Response Time (Latency)

- Example: Consider 2 computer, the execution time for a Task by using 2 computers are:

- Computer A: 8 s
- Computer B: 12 s

Which one is faster in terms of response time?

Answer

Performance is inversely proportional to execution time:

$$Performance = \frac{1}{Execution\ Time}$$

Therefore:

$$\frac{Performance_A}{Performance_B} = \frac{Time_B}{Time_A} = \frac{12}{8} = 1.5$$

Computer A is **1.5× faster** than Computer B.

Performance of a Computer

Throughput (Bandwidth)

Example: Consider the performance of 2 systems:

- System A
 - 1 request: 20 ms
 - 100 requests/s
- System B
 - 1 request: 25 ms
 - 150 requests/s

Questions

- Which has better latency?
- Which has better throughput?

Answer

- A has better latency.
- B has better throughput.

“Faster” must be defined using a metric.

Performance of a Computer

CPU execution time

- **CPU execution time is determined by three major factors:**
 - Instruction Count (IC)
 - Cycles Per Instruction (CPI)
 - Clock Cycle Time

$$CPU\ Time = Instruction\ Count \times CPI \times Clock\ Cycle\ Time$$

Since:

$$Clock\ Cycle\ Time = \frac{1}{Clock\ Rate}$$

then:

$$CPU\ Time = \frac{Instruction\ Count \times CPI}{Clock\ Rate}$$

- Performance depends on the interaction between software, ISA, and processor implementation.

Performance of a Computer

IC, CPI, and Clock

- **Instruction Count (IC)**
 - Number of instructions executed by a program
 - Influenced by:
 - algorithm
 - compiler
 - ISA
- **Cycles Per Instruction (CPI)**
 - Average number of clock cycles required per instruction
 - Influenced mainly by:
 - processor implementation
 - instruction mix
 - pipeline stalls
 - memory behavior
- **Clock Rate**
 - Number of clock cycles per second
 - Usually measured in Hz, MHz, or GHz
- **Important**
 - Higher clock rate does not automatically mean higher performance.

Performance of a Computer

Clock Cycle vs Clock Rate

- **Clock cycle**
 - One timing interval of the processor
- **Clock cycle time**

$$T_{clock} = \frac{1}{\text{Clock Rate}}$$

- **Example**
 - For a 2 GHz processor:

$$T_{clock} = \frac{1}{2 \times 10^9} = 0.5 \text{ ns}$$

- For a 4 GHz processor:

$$T_{clock} = 0.25 \text{ ns}$$

- **Key distinction**
 - Clock rate → cycles per second
 - Clock cycle time → seconds per cycle
 - Total clock cycles → number of cycles required by the program

Performance of a Computer

Total Clock Cycles

- The total number of processor cycles is:

$$CPU \text{ Clock Cycles} = \text{Instruction Count} \times CPI$$

- Therefore:

$$CPU \text{ Time} = CPU \text{ Clock Cycles} \times \text{Clock Cycle Time}$$

- **Example:** Given:

- IC = 1,000,000 instructions
- CPI = 2
- Clock rate = 2 GHz

Then:

$$Clock \text{ Cycles} = 1,000,000 \times 2 = 2,000,000$$

$$CPU \text{ Time} = \frac{2,000,000}{2 \times 10^9} = 1 \text{ ms}$$

Performance of a Computer

Average CPI

- Different instruction types may require different numbers of cycles.

$$CPI_{avg} = \frac{\sum (CPI_i \times IC_i)}{\sum IC_i}$$

- Example**

<i>Instruction Type</i>	<i>Instruction Count</i>	<i>CPI</i>
<i>ALU</i>	500	1
<i>Load/Store</i>	300	2
<i>Branch</i>	200	3

- Total cycles:

$$500(1) + 300(2) + 200(3) = 1700$$

- Total instructions:

$$1000$$

- Average CPI:

$$CPI_{avg} = 1.7$$

- CPI depends on the instruction mix.

Performance of a Computer

Performance Comparison Example

- Processor A:
 - $IC = 1.0 \times 10^9$
 - $CPI = 1.5$
 - Clock rate = 3 GHz
- Processor B:
 - $IC = 1.2 \times 10^9$
 - $CPI = 1.0$
 - Clock rate = 2.5 GHz
- For A:

$$CPU\ Time_A = \frac{1.0 \times 10^9 \times 1.5}{3 \times 10^9} = 0.5s$$

- For B:

$$CPU\ Time_B = \frac{1.2 \times 10^9 \times 1.0}{2.5 \times 10^9} = 0.48s$$

- **Conclusion**
 - B has a lower clock rate
 - But B executes the program faster
- **Clock frequency alone is not a valid performance metric.**

Performance of a Computer

What Can Change Performance?

$$CPU\ Time = IC \times CPI \times Clock\ Cycle\ Time$$

- **Algorithm**
 - May affect IC
- **Programming language / compiler**
 - May affect IC
 - May affect instruction mix
- **ISA**
 - May affect IC and CPI
- **Microarchitecture**
 - Mainly affects CPI and clock cycle time
- **Memory system**
 - Can strongly affect CPI

Computer performance is a system-level property

Benchmarking

Benchmarking

- A benchmark is a program or workload used to measure computer performance.
- **Why not simply compare clock rate?**
- Because performance depends on:
 - application behavior
 - instruction count
 - CPI
 - memory accesses
 - compiler
 - processor implementation
- **Therefore:** Performance should be measured using representative workloads.

Benchmarking

Types of Benchmarks

- **Real applications**
 - Compilers
 - Web servers
 - Scientific applications
 - Media processing
- **Kernels**
 - Small portions of real applications
 - Focus on one important operation
- **Synthetic benchmarks**
 - Artificial programs designed to reproduce certain workload characteristics
- **Benchmark suites**
 - Collections of programs
 - Used to provide a broader view of system performance
- Examples:
 - **SPEC CPU**: processor and memory-intensive workloads, widely used for CPU performance evaluation
 - **TPC**: database and transaction-processing workloads
 - **MLPerf**: machine-learning workloads
 - **CoreMark**: commonly used in embedded systems

Benchmarking

Good Benchmarks

- A good benchmark should be:
 - Representative
 - Repeatable
 - Relevant to the target application
 - Measurable
 - Difficult to manipulate unfairly
- **Questions to ask**
 - Does the benchmark resemble the real workload?
 - Is one benchmark enough?
 - Are the systems compiled using comparable settings?
 - Are we measuring latency or throughput?

Benchmark Pitfalls

- Be careful when comparing systems using:
 - only one application
 - only clock frequency
 - peak theoretical performance
 - different compiler optimizations
 - different workloads
 - vendor-specific benchmark settings

Amdahl's Law

Amdahl's Law

- Suppose only part of a program can be improved. Let:
 - f = fraction of execution time affected
 - S = speedup of the improved part
- Then the overall speedup is:

$$Speedup_{overall} = \frac{1}{(1 - f) + \frac{f}{S}}$$

This is **Amdahl's Law**.

- **f : Fraction of the original execution time that can be improved**
 - It represents the portion of the program affected by the optimization.
 - $0 \leq f \leq 1$
 - It is measured using the **original execution time**, before optimization.
 - It is **not** the percentage of source-code lines or instructions.
- **S : Speedup of the improved portion**
 - It represents how many times faster the improved part becomes.
 - If $S = 2$, that part takes half of its original time.
 - If $S = 4$, that part takes one quarter of its original time.
 - S applies only to the portion represented by f , not to the entire program.

Amdahl's Law

Amdahl's Law

- **Example:**

- Suppose a program originally takes **100 seconds**. 40 seconds can be optimized:

$$f = \frac{40}{100} = 0.4$$

- The optimized part becomes 4 times faster:

$$S = 4$$

- Therefore, the optimized 40-second portion now takes:

$$\frac{40}{4} = 10 \text{ seconds}$$

- The remaining 60 seconds are unchanged.

- So the new execution time is:

$$T_{new} = 60 + 10 = 70 \text{ seconds}$$

and the overall speedup is:

$$Speedup_{overall} = \frac{100}{70} \approx 1.43$$

- f tells us how much of the original execution time can benefit from the optimization, while S tells us how much faster that particular portion becomes.

Amdahl's Law

The Limit of Optimization

- Even if one part of a program is made infinitely fast, the overall program can only be accelerated up to a certain limit because the rest of the program is unchanged.
- If the improved part becomes infinitely fast:

$$S \rightarrow \infty$$

So the maximum possible speedup becomes:

$$Speedup_{max} = \frac{1}{1 - f}$$

This is the optimization limit.

Amdahl's Law

The Limit of Optimization

- **Example:** Suppose a program originally takes **100 s**:
 - 40 s can be improved
 - 60 s cannot be improved

So:

$$f = 0.4$$

- Even if the 40 s part becomes infinitely fast, its execution time approaches:

$$\frac{40}{\infty} \approx 0$$

- But the other 60 s still remains. Therefore:

$$T_{new,min} = 60 \text{ s}$$

- Hence:

$$Speedup_{max} = \frac{100}{60} = 1.67$$

- So no matter how powerful the optimization is, the entire program can never become more than **1.67× faster**.
- The important idea is that the **unoptimized portion becomes the bottleneck**.